Advanced Dynamic Pricing ML System (Industry-Level Guide)

This document is a deep, industry-level guide for building a real dynamic pricing system. It goes beyond basic ML and introduces proper time-series handling, leakage prevention, price optimization, and realistic backtesting. This is designed to reflect how real companies build pricing systems.

Step 1: Why Random Split is WRONG

Reason: In time-based problems, random splitting leaks future information into training. This creates unrealistic performance.

Code:

```
# ■ WRONG train_test_split(X, y) # ■ CORRECT (time-based split) df = df.sort_values("date") train = df[df["date"] < "2022-10-01"] test = df[df["date"] >= "2022-10-01"]
```

Step 2: Define Features and Target

Reason: We predict demand. Pricing will be derived from predicted demand.

Code:

```
X_train = train.drop(["units_sold", "date"], axis=1) y_train = train["units_sold"] X_test = test.drop(["units_sold", "date"], axis=1) y_test = test["units_sold"]
```

Step 3: Why NOT Normalize?

Reason: Tree-based models like XGBoost and LightGBM do not require normalization because they split based on thresholds.

Code:

```
# No scaling required
```

Step 4: Train XGBoost Properly

Reason: Use more realistic parameters instead of defaults.

Code:

```
import xgboost as xgb model = xgb.XGBRegressor( n_estimators=300, learning_rate=0.05, max_depth=8, subsample=0.8, colsample_bytree=0.8 ) model.fit(X_train, y_train)
```

Step 5: Hyperparameter Tuning (Better Approach)

Reason: GridSearch is slow. Use randomized search or Optuna in real systems.

Code:

```
from sklearn.model_selection import RandomizedSearchCV params = { "max_depth": [4,6,8], "learning_rate": [0.01, 0.05, 0.1] } search = RandomizedSearchCV(model, params, n_iter=5) search.fit(X_train, y_train)
```

Step 6: Feature Importance (Understanding Model)

Reason: We must verify model behavior aligns with business logic.

Code:

```
import matplotlib.pyplot as plt xgb.plot_importance(model) plt.show()
```

Step 7: REAL Price Optimization (Core Idea)

Reason: Instead of simple rules, simulate multiple prices and choose the best.

Code:

```
def find_best_price(row): prices = [row["price"] * i for i in [0.8, 0.9, 1.0, 1.1, 1.2]] best_price = row["price"] best_revenue = 0 for p in prices: temp = row.copy() temp["price"] = p demand = model.predict([temp.drop("units_sold")])[0] revenue = demand * p if revenue > best_revenue: best_revenue = revenue best_price = p return best_price
```

Step 8: Apply Price Optimization

Reason: This simulates decision-making like real systems.

Code:

```
df["optimal_price"] = df.apply(find_best_price, axis=1)
```

Step 9: Backtesting (REAL WAY)

Reason: Simulate day-by-day decisions instead of batch predictions.

Code:

```
revenues = [] for i in range(len(test)): row = test.iloc[i] price = find_best_price(row) revenue = price * row["units_sold"] revenues.append(revenue)
```

Step 10: Compare with Baseline

Reason: Compare against rule-based system.

Code:

```
baseline = test["dynamic_price"] * test["units_sold"] ml = sum(revenues) lift = (ml - baseline.sum()) / baseline.sum() * 100 print("Lift:", lift)
```

Step 11: Overfitting Check

Reason: If train error << test error → overfitting.

Code:

```
train_pred = model.predict(X_train) test_pred = model.predict(X_test)
```

Step 12: SHAP Explainability

Reason: Understand WHY model makes decisions.

Code:

```
import shap explainer = shap.Explainer(model) shap_values = explainer(X_test) shap.plots.beeswarm(shap_values)
```

Step 13: Real Insight Extraction

Reason: Students must interpret results like:

Code:

```
# Example: # 'Price has strongest negative correlation with demand' # 'Inventory pressure drives pricing decisions'
```